SISTEMAS OPERATIVOS

Aula Teórica 15 - EXERCÍCIOS

Daniel Castro
2025/2026

UAlg
UNIVERSIDADE DO ALGARVE

Exercícios (1)

Exercício 1 - Considere um dado programa, com várias threads, que tem 3 versões que se executam num computador uniprocessador com sistema operativo do tipo Unix e cuja funcionalidade não implica nenhuma operação de I/O:

- **versão V1 em que cada thread é suportado por um processo,**
- **versão V2 em que cada thread é suportado por uma tarefa real, e**
- **versão V3 em que cada thread é suportado por uma pseudo-tarefa (biblioteca no espaço user).**

Note que as 3 versões têm exatamente a mesma funcionalidade.

a) Como compara as versões V1 e V2 no que respeita à robustez do programa? Na sua resposta considere, por exemplo, que uma das threads faz uma divisão por zero.

Exercícios (1)

Exercício 1 - Considere um dado programa, com várias threads, que tem 3 versões que se executam num computador uniprocessador com sistema operativo do tipo Unix e cuja funcionalidade não implica nenhuma operação de I/O:

- **versão V1 em que cada thread é suportado por um processo,**
- **versão V2 em que cada thread é suportado por uma tarefa real, e**
- **versão V3 em que cada thread é suportado por uma pseudo-tarefa (biblioteca no espaço user).**

Note que as 3 versões têm exatamente a mesma funcionalidade.

a) Como compara as versões V1 e V2 no que respeita à robustez do programa? Na sua resposta considere, por exemplo, que uma das threads faz uma divisão por zero.

A versão V1 é mais robusta, uma vez que apenas o processo onde ocorre o erro é terminado, continuando as restantes tarefas a executar-se.
Na versão V2 o processo (que inclui todas as tarefas) é terminado.
No caso de uma divisão por zero, em V1, apenas a thread associado ao processo onde ocorre a divisão por zero termina (e todos as outras threads continuam a sua execução normal) enquanto que em V2, todos as threads são terminados.

Exercícios (2)

b) Como compara as versões V1 e V3 no que respeita à robustez do programa? Na sua resposta considere, por exemplo, que uma das threads faz uma divisão por zero.

Resposta idêntica à anterior.

c) Como compara as versões V1 e V2 no que respeita à rapidez de execução do programa?

V1 será mais lento, pois a criação de processos e comutação entre processos é mais lenta que a criação/comutação entre tarefas, dado que os processos não partilham o mesmo contexto.

d) Como compara as versões V1 e V3 no que respeita à rapidez de execução do programa?

Resposta idêntica à anterior.

e) Considere agora que o programa é executado numa máquina com dois processadores. Compare o desempenho das versões V2 e V3.
A versão V2 será mais rápida pois diferentes tarefas poderão executar-se concorrentemente nos dois processadores. Na versão V3, todas as pseudo-tarefas irão partilhar um único processador.

Exercícios (3)

Exercício 2 - Considere o seguinte excerto de um programa Unix (por simplicidade, omitem-se verificações de erros):

```
1.  int a; /* variável global */
2.  main() {
3.      a = 0;
4.      if (fork() == 0) {
5.          a++;
6.          printf("%d", a);
7.          exit(0);
8.      }
9.      else {
10.         pid = wait(&estado);
11.     }
12.     printf("%d", a);
13. }
```

a) Qual a saída deste programa? Justifique.

Exercícios (3)

Exercício 2 - Considere o seguinte excerto de um programa Unix (por simplicidade, omitem-se verificações de erros):

```
1.  int a; /* variável global */
2.  main() {
3.      a = 0;
4.      if (fork() == 0) {
5.          a++;
6.          printf("%d", a);
7.          exit(0);
8.      }
9.      else {
10.         pid = wait(&estado);
11.     }
12.     printf("%d", a);
13. }
```

a) Qual a saída deste programa? Justifique.

Saída: '10'.

O primeiro algarismo ('1') é impresso após a criação do processo filho e depois do incremento unitário à variável 'a', que depois é impressa (linha 6).

O segundo algarismo ('0') é impresso pelo processo pai depois do processo filho terminar (linha 12).

*É de notar que **os processos não partilham o mesmo espaço de endereçamento**, razão pela qual a variável 'a' possui valores distintos nos dois processos.*

Exercícios (4)

b) - Modifique-o de modo a obter outra sequência. Por simplicidade, omita verificações de erros. Justifique.
Várias respostas possíveis. Possível resposta.

```
1.  int a; /* variável global */
2.  main() {
3.      a = 0;
4.      if (fork() == 0) {
5.          a++;
6.          printf("%d", a);
7.          /* exit(0);  */
8.      }
9.      else {
10.         pid = wait(&estado);
11.     }
12.     printf("%d", a);
13. }
```

Saída:

Exercícios (4)

b) - Modifique-o de modo a obter outra sequência. Por simplicidade, omita verificações de erros. Justifique.
Várias respostas possíveis. Possível resposta.

```
1.  int a; /* variável global */
2.  main() {
3.     a = 0;
4.     if (fork() == 0) {
5.        a++;
6.        printf("%d", a);
7.        /* exit(0);  */
8.     }
9.     else {
10.       pid = wait(&estado);
11.    }
12.    printf("%d", a);
13. }
```

Saída: '110'. Os primeiros dois '1's são impressos pelo filho (linhas 6 e 12). O '0' é impresso pelo pai (linha 12). O valor da variável 'a' no processo pais é zero pois apenas o filho incrementa a variável (linha 5). Uma vez que os processos não partilham o mesmo espaço de endereçamento, um incremento de uma variável no filho não tem qualquer efeito no processo do pai.

Exercícios (5)

Exercício 3 - Para cada uma das seguintes afirmações, indique se esta pode ou não ser um motivo válido para a utilização de múltiplas tarefas reais (tarefas-núcleo) num programa que corre num sistema operativo Unix, e explique porquê:

a) Ter múltiplas threads com espaços de endereçamento separados no mesmo programa.

Não; ao utilizar tarefas reais, o espaço de endereçamento é comum a todas as tarefas do mesmo processo.

b) Poder sobrepor a execução de instruções de E/S com a utilização do CPU por outras partes do mesmo programa.

Sim, ao utilizar tarefas reais, enquanto uma ou mais tarefas podem estar a tratar eventos de E/S, outras poderão estar a utilizar CPU.

c) Executar dois programas diferentes (correspondentes a diferentes ficheiros executáveis) em paralelo.

Não; tarefas reais não são necessárias para atingir paralelismo entre dois processos diferentes. As tarefas reais podem apenas trazer paralelismo dentro do mesmo processo (i.e., várias threads em paralelo).

Exercícios (6)

Exercício 4 - Considere o seguinte excerto de um programa que usa signals em Unix (por simplicidade, omitem-se verificações de erros):

```
1. void trata_signal(int num) {
2.     signal(SIGINT, trata_signal);
3.     printf("Operação negada.");
4. }
5.
6. int main(void) {
7.     if (signal(SIGINT, trata_signal) == SIG_ERR)
8.         {printf("Erro no signal");}
9.     for (;;) {sleep(10);}
10.    exit(0);
11.}
```

a) Explique o comportamento deste programa.

O programa começa por instalar uma rotina de tratamento para o sinal 'SIGINT' usando a chamada de sistema "signal" e fazendo o teste devido para verificar o retorno da chamada de sistema. De seguida, o programa entra num ciclo "for" sem fim onde, a cada iteração, o programa suspende a sua execução durante 10 segundos (utilizando a chamada de sistema "sleep").
A rotina de tratamento do sinal "SIGINT", se chamada, re-instala o sinal e imprime uma mensagem para a output. É de notar que o sinal "SIGINT" pode ser gerado através da combinação das teclas CTRL+C.

b) Identifique todas as funções do sistema operativo e da biblioteca stdio que são utilizadas no programa e explique o que faz cada uma.

"printf" – imprime carateres para o output do processo;
"signal" – instala rotinas de tratamento de sinais;
"sleep" – suspende a execução da tarefa atual durante um período de tempo;
"exit" – termina a execução do processo.

c) De que forma um processo que corra este programa pode ser terminado? Justifique.

O processo pode ser terminado enviando um sinal cujo efeito sobre o processo é terminá-lo. Por exemplo, é possível utilizar a chamada de sistema "kill" com o sinal "SIGTERM" para terminar este processo.

Exercícios (8)

Exercício 5 - Considere a chamada de sistema "kill" em Unix.

a) Explique o que faz esta função.

A chamada de sistema "kill" tem como propósito o envio de sinais a processos.

b) Esta função necessita de efetuar algumas validações de segurança. Explique quais e qual o motivo da sua existência.

Uma vez que a chamada de sistema "kill" pode interferir com outros processos, é importante garantir que, por exemplo, um utilizador sem privilégios de superutilizador não consiga interferir com processos do sistema (que estão associados ao superutilizador do sistema) ou com processos de outros utilizadores. Desta forma, usando a função de sistema "kill", apenas é possível enviar sinais a processos do mesmo UID. Apenas processos com privilégios de superutilizador podem ultrapassar esta proteção.

c) O nome desta primitiva é enganador. Explique porquê.

É enganador pois a função não mata o processo, apenas envia sinais.
A possível motivação para o nome resulta da implementação por omissão do tratamento da maioria dos sinais, que resulta no término do processo.

Exercícios (9)

Exercício 6 - Considere o código do seguinte programa Unix (por simplicidade, omitem-se verificações de erros):

```
1.  #include <stdio.h>
2.  int a;
3.  fn() {
4.        a++;
5.        printf("X %d\n", a);
6.        exit(0);
7.       }
8.
9.  main(int argc, char **argv) {
10.      int val;
11.      a = 0;
12.      printf("Y %d\n", a);
13.      if (fork() == 0) {fn();}
14.      wait(&val);
15.      printf("Z %d\n", a);
16. }
```

a) Qual a saída deste programa? Se houver mais do que uma possibilidade, indique todas as possíveis saídas.

Exercícios (9)

Exercício 6 - Considere o código do seguinte programa Unix (por simplicidade, omitem-se verificações de erros):

```
1.  #include <stdio.h>
2.  int a;
3.  fn() {
4.        a++;
5.        printf("X %d\n", a);
6.        exit(0);
7.       }
8.
9.  main(int argc, char **argv) {
10.      int val;
11.      a = 0;
12.      printf("Y %d\n", a);
13.      if (fork() == 0) {fn();}
14.      wait(&val);
15.      printf("Z %d\n", a);
16. }
```

a) Qual a saída deste programa? Se houver mais do que uma possibilidade, indique todas as possíveis saídas.

Saída possível 1:

Y = 0

X =1

Z =0

Saída possível 2 (se o "fork" falhar):

Y = 0

Z = 0

Exercícios (10)

b) Escreva um novo programa que seja equivalente (no sentido de executar a mesma sequência de código, apesar de não ter necessariamente a mesma saída), exceto no seguinte aspeto: na linha 13 deve lançar uma nova tarefa em vez de um novo processo. Deve utilizar a interface POSIX para todas as funções relacionadas com as tarefas. Tenha o cuidado de atualizar as restantes funções do sistema operativo que são invocadas neste código em conformidade. Por simplicidade, omita verificações de erros.

Exercícios (10)

b) Escreva um novo programa que seja equivalente (no sentido de executar a mesma sequência de código, apesar de não ter necessariamente a mesma saída), exceto no seguinte aspeto: na linha 13 deve lançar uma nova tarefa em vez de um novo processo. Deve utilizar a interface POSIX para todas as funções relacionadas com as tarefas. Tenha o cuidado de atualizar as restantes funções do sistema operativo que são invocadas neste código em conformidade. Por simplicidade, omita verificações de erros.

```
1.  #include <stdio.h>
2.  int a;
3.  void *fn(void *arg) {
4.      a++;
5.      printf("X %d\n", a);
6.      return 0;
7.  }
8.
9.  main(int argc, char **argv) {
10.     pthread_t tid;
11.     a = 0;
12.     printf("Y %d\n", a);
13.     pthread_create(&tid, NULL, fn, NULL);
14.     pthread_join(tid, NULL);
15.     printf("Z %d\n", a);
16. }
```

c) Qual a saída do novo programa?

Exercícios (11)

c) Qual a saída do novo programa?

Saída possível 1:
Y = 0
X = 1
Z = 1
Saída possível 2 (se o "pthread_create" falhar):
Y = 0
Z = 0

d) Ordene em termos de velocidade de comutação, da mais rápida para a mais lenta, as seguintes possibilidades para a execução deste programa:
Usando tarefas núcleo;
Usando pseudotarefas;
Usando processos;

Exercícios (11)

c) Qual a saída do novo programa?

Saída possível 1:
Y = 0
X = 1
Z = 1
Saída possível 2 (se o "pthread_create" falhar):
Y = 0
Z = 0

d) Ordene em termos de velocidade de comutação, da mais rápida para a mais lenta, as seguintes possibilidades para a execução deste programa:
Usando tarefas núcleo;
Usando pseudotarefas;
Usando processos;

Pseudotarefas, tarefas núcleo, processos.

e) Para a resposta anterior, explique a razão principal para a escolha relativa entre as tarefas núcleo e as pseudotarefas.

Exercícios (11)

c) Qual a saída do novo programa?

Saída possível 1:
Y = 0
X = 1
Z = 1
Saída possível 2 (se o "pthread_create" falhar):
Y = 0
Z = 0

d) Ordene em termos de velocidade de comutação, da mais rápida para a mais lenta, as seguintes possibilidades para a execução deste programa:
Usando tarefas núcleo;
Usando pseudotarefas;
Usando processos;

Pseudotarefas, tarefas núcleo, processos.

e) Para a resposta anterior, explique a razão principal para a escolha relativa entre as tarefas núcleo e as pseudotarefas.

A principal razão prende-se com o facto da comutação de tarefas reais implicar mais custo do que a comutação de pseudotarefas pois no primeiro caso é preciso passar de modo de utilizador para modo núcleo e vice versa enquanto que no segundo caso a comutação apenas em modo de utilizador.

Exercícios (12)

Exercício 7 - Suponha que num sistema operativo estão a correr dois processos. O processo P2 é CPU intensivo (não realizando qualquer operações de E/S) e o processo P1 executa o seguinte código:

```
main() {
   while(!terminado) {
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      efetuaES();        // Operação de E/S com duração de 2 unidades de tempo
                         // (quando chamada, bloqueia imediatamente durante 2 unidades de tempo)
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
   }
}
```

Os processos têm as seguintes características:

Processo	Prioridade Base	Instante de Início de Execução	Tempo Total de CPU
P1	10	0	6
P2	11	3	3

Nota: a coluna Tempo Total de CPU denota o tempo que o processo irá consumir durante toda a sua execução.

Exercícios (13)

Suponha que:
- O algoritmo de escalonamento utilizado é preemptivo, com prioridades dinâmicas (em que um valor numérico de prioridade mais elevado corresponde a um processo mais prioritário);
- A prioridade de um processo começa por ser sua prioridade base, sofrendo um aumento de três unidades sempre que o processo sai do estado bloqueado;
- A prioridade do processo diminui uma unidade após cada quantum (time-slice) em que o processo tenha estado em execução, até atingir novamente a prioridade base:
- Se dois processos tiverem igual prioridade, o escalonador escolhe o que não é executado há mais tempo;
- A duração do quantum é de uma unidade de tempo;

Preencha a seguinte tabela, indicando, para cada time-slice, qual o estado de cada processo do sistema, e a respetiva prioridade no início do quantum. Utilize a seguinte notação: Letra E – Em execução; Letra B – Bloqueado; Letra V – Executável; Entrada em branco – processo não está ativo no sistema (não iniciou ou já terminou).

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
P1															
P2															

Note que na solução correta não é necessário utilizar mais espaço do que o fornecido.

Exercícios (14)

Processo	Prioridade Base	Instante de Início de Execução	Tempo Total de CPU
P1	10	0	6
P2	11	3	3

O processo P1 executa o seguinte código:

```
main() {
   while(!terminado) {
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      efetuaES();        // Operação de E/S com duração de 2 unidades de tempo
                         // (quando chamada, bloqueia imediatamente durante 2 unidades de tempo)
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
   }
}
```

O processo P2 é CPU intensivo (não realizando qualquer operações de E/S);

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
P1	E (10)	E (10)	E>B (10)												
P2															

Note que na solução correta não é necessário utilizar mais espaço do que o fornecido.

Exercícios (14)

Processo	Prioridade Base	Instante de Início de Execução	Tempo Total de CPU
P1	10	0	6
P2	11	3	3

O processo P1 executa o seguinte código:

```
main() {
   while(!terminado) {
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      efetuaES();        // Operação de E/S com duração de 2 unidades de tempo
                         // (quando chamada, bloqueia imediatamente durante 2 unidades de tempo)
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
   }
}
```

O processo P2 é CPU intensivo (não realizando qualquer operações de E/S);

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
P1	E (10)	E (10)	E>B (10)	B (10)											
P2				E (11)											

Note que na solução correta não é necessário utilizar mais espaço do que o fornecido.

Exercícios (14)

Processo	Prioridade Base	Instante de Início de Execução	Tempo Total de CPU
P1	10	0	6
P2	11	3	3

O processo P1 executa o seguinte código:

```
main() {
   while(!terminado) {
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      efetuaES();        // Operação de E/S com duração de 2 unidades de tempo
                         // (quando chamada, bloqueia imediatamente durante 2 unidades de tempo)
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
   }
}
```

O processo P2 é CPU intensivo (não realizando qualquer operações de E/S);

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
P1	E (10)	E (10)	E>B (10)	B (10)	E (13)	E (12)	V (11)								
P2				E (11)	V (11)	V (11)	E (11)								

Note que na solução correta não é necessário utilizar mais espaço do que o fornecido.

Exercícios (14)

Processo	Prioridade Base	Instante de Início de Execução	Tempo Total de CPU
P1	10	0	6
P2	11	3	3

O processo P1 executa o seguinte código:

```
main() {
   while(!terminado) {
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
      efetuaES();        // Operação de E/S com duração de 2 unidades de tempo
                         // (quando chamada, bloqueia imediatamente durante 2 unidades de tempo)
      trabalhoCPU();     // Requer uma unidade de tempo de CPU
   }
}
```

O processo P2 é CPU intensivo (não realizando qualquer operações de E/S);

	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14
P1	E (10)	E (10)	E>B (10)	B (10)	E (13)	E (12)	V (11)	E (11)	E>B (10)	B (10)	E (13)				
P2				E (11)	V (11)	V (11)	E (11)	V (11)	E (11)						

Note que na solução correta não é necessário utilizar mais espaço do que o fornecido.

Nota: nas time-slices 2 e 8, são representadas as transições do estado E para o estado B dado que, no início de cada uma dessas time-slices, o processo está em execução; no entanto, ao chamar a função efetuaES, este é bloqueado de seguida. Pressupomos que o tempo dispendido no bloqueio já insere a paragem por V para poder executar de seguida.

Exercícios (15)

Exercício 8 – No âmbito das soluções que asseguram a exclusão mútua, diga em que consiste a noção de míngua (starvation). Como se devem comportar as soluções corretas no que diz respeito à míngua?

A noção de míngua (ou starvation) refere-se à situação onde uma ou mais tarefas, que pretendem aceder a um recurso, não o conseguem, sistematicamente sendo ultrapassadas por outras tarefas que lhe acedem (mesmo que estejam há menos tempo à espera de entrar). Uma solução correta relativamente à míngua deve garantir que todas as tarefas têm igual oportunidade de acesso ao recurso em causa, não havendo preferências ou prioridades na seleção de tarefas a entrar na secção crítica.

Exercício 9 - Diga o que entende por espera ativa numa solução para o problema da secção crítica.

Espera ativa numa solução para o problema da secção crítica significa que um ou mais threads estão continuamente a testar uma condição de entrada na secção crítica e essa condição só se alterará se a thread deixar de efetuar esse mesmo teste. A utilização de espera ativa é muito comum em soluções algorítmicas para exclusão mútua (e é uma desvantagem clara relativamente a soluções com base em objetos do sistema operativo que não ocupam o processador de forma inútil enquanto a condição de espera não varia).

Exercícios (16)

Exercício 10 - Considere um sistema operativo em que, durante a sequência de teste e modificação de uma variável partilhada (trinco), a exclusão mútua é conseguida através da inibição das interrupções. Diga qual a desvantagem desta solução.

Existem várias desvantagens:

não funciona em multiprocessadores;
só funciona dentro do núcleo;
só é aceitável com secções críticas muito pequenas;
torna o sistema vulnerável a bugs dentro da secção crítica.

Exercícios (17)

Exercício 11 - Considere uma garagem na qual existem N lugares de estacionamento. A garagem tem várias entradas, sendo o acesso gerido por uma aplicação central com múltiplas tarefas. Cada tarefa é responsável por interagir com um terminal em cada entrada da garagem. Cada carro com acesso ao parque é identificado por um inteiro.
Cada condutor que se aproxima de uma entrada executa a função ocupar_lugar, implementada como de seguida, para reservar um lugar antes de entrar, e chama liberta_lugar quando sai da garagem. O vetor lugares[N] mantém o estado de cada lugar; a cada momento, cada entrada no vetor pode conter o valor LIVRE ou o identificador do carro que tem direito a ocupar o lugar.

```
int lugares[N] = {LIVRE, ..., LIVRE}; /* todos os elementos estão LIVRES */

1. int ocupa_lugar(int carId) {
2.   int i;
3.   for (i=0; i<N; i++) {
4.     if (lugares[i] == LIVRE) {
5.       lugares[i] = carId;
6.       return i;
7.     }
8.   }
9.   return -1;
10. }

11. void liberta_lugar(int carId) {
12.   int i;
13.   for (i=0; i<N; i++) {
14.     if (lugares[i] == carId) {
15.       lugares[i] = LIVRE;
16.       return;
17.     }
18.   }
19. }
```

Exercícios (18)

a) Após a inauguração do sistema, verificaram-se situações de conflitos entre condutores que reclamaram que, tendo acedido à garagem concorrentemente por entradas diferentes, lhes tinha sido atribuído o mesmo lugar. Analisando a implementação acima, indique a razão deste erro. Ilustre-a com um cenário concreto de execução que leve a esta situação indesejada.

```
int lugares[N] = {LIVRE, ..., LIVRE}; /* todos os elementos estão LIVRES */
1. int ocupa_lugar(int carId) {
2.   int i;
3.   for (i=0; i<N; i++) {
4.     if (lugares[i] == LIVRE) {
5.       lugares[i] = carId;
6.       return i;
7.     }
8.   }
9.   return -1;
10. }

11. void liberta_lugar(int carId) {
12.   int i;
13.   for (i=0; i<N; i++) {
14.     if (lugares[i] == carId) {
15.       lugares[i] = LIVRE;
16.       return;
17.     }
18.   }
19. }
```

Duas tarefas chamam ocupa_lugar concorrentemente, observando ambas o lugar 0 livre (linha 4, primeira iteração do ciclo) antes de alguma marcar o lugar como ocupado (linha 5), logo ambas ocupam esse lugar.

b) Proponha uma correção ao programa. Na sua solução não deve recorrer a espera ativa.

Exercícios (19)

b) Proponha uma correção ao programa. Na sua solução não deve recorrer a espera ativa.

```
int lugares[N] = {LIVRE, ..., LIVRE}; /* todos os elementos estão LIVRES */
 1. int ocupa_lugar(int carId) {
 2.   int i;
 3.   for (i=0; i<N; i++) {
 4.     if (lugares[i] == LIVRE) {
 5.       lugares[i] = carId;
 6.       return i;
 7.     }
 8.   }
 9.   return -1;
 10. }

 11. void liberta_lugar(int carId) {
 12.   int i;
 13.   for (i=0; i<N; i++) {
 14.     if (lugares[i] == carId) {
 15.       lugares[i] = LIVRE;
 16.       return;
 17.     }
 18.   }
 19. }
```

```
int lugares[N] = {LIVRE, ..., LIVRE}; /* todos os elementos estão LIVRES */
trinco t;

1. int ocupa_lugar(int carId) {
2.   int i;
3.   fechar(t);
4.   for (i=0; i<N; i++) {
5.     if (lugares[i] == LIVRE) {
6.       lugares[i] = carId;
7.       abrir(t);
8.       return i;
9.     }
10.   }
11.   abrir(t);
12.   return -1;
13. }

14. void liberta_lugar(int carId) {
15.   int i;
16.   fechar(t);
17.   for (i=0; i<N; i++) {
18.     if (lugares[i] == carId) {
19.       lugares[i] = LIVRE;
20.       abrir(t);
21.       return;
22.     }
23.   }
24.   abrir(t);
25. }
```

Exercícios (20)

c) Sempre que a função ocupa_lugar é chamada mas a garagem está cheia, a função deve esperar até que um lugar esteja finalmente disponível. Partindo da solução anterior, estenda ambas as funções para cumprir este novo requisito usando variáveis de condição.

```
int lugares[N] = {LIVRE, ..., LIVRE}; /* todos os elementos estão LIVRES */
trinco t;

1. int ocupa_lugar(int carId) {
2.   int i;
3.   fechar(t);
4.   for (i=0; i<N; i++) {
5.     if (lugares[i] == LIVRE) {
6.       lugares[i] = carId;
7.       abrir(t);
8.       return i;
9.     }
10.  }
11.  abrir(t);
12.  return -1;
13. }

14. void liberta_lugar(int carId) {
15.   int i;
16.   fechar(t);
17.   for (i=0; i<N; i++) {
18.     if (lugares[i] == carId) {
19.       lugares[i] = LIVRE;
20.       abrir(t);
21.       return;
22.     }
23.   }
24.   abrir(t);
25. }
```

```
int lugares[N] = {LIVRE, ..., LIVRE}; /* todos os elementos estão LIVRES */
int livres = N;
trinco t;
varcond condLugar;

1. int ocupa_lugar(int carId) {
2.     int i;
3.     fechar(t);
4.     while (livres == 0)
5.         wait(condLugar, t);
6.         for (i=0; i<N; i++) {
7.             if (lugares[i] == LIVRE) {
8.                 lugares[i] = carId;
9.                 livres --;
10.                abrir(t);
11.                return i;
12.            }
13.        }
14.        abrir(t); //Nunca deverá chegar aqui
15.        return -1;
16. }

17.   void liberta_lugar(int carId) {
18.     int i;
19.     fechar(t);
20.     for (i=0; i<N; i++) {
21.         if (lugares[i] == carId) {
22.             lugares[i] = LIVRE;
23.             livres ++;
24.             signal(condLugar);
25.         abrir(t);
26.         return;
27.       }
28.     }
29.    abrir(t);
30. }
```

Exercícios (21)

d) Suponha que uma tarefa T1 chamou ocupa_lugar mas a garagem estava cheia, o que colocou T1 à espera. Quando, de seguida, uma tarefa T2 chamou liberta_lugar. A sua solução permite que uma terceira tarefa, T3, que chame ocupa_lugar depois de T1 se ter colocado em espera possa ganhar um lugar antes de T1?

Sim. Quando T2, ao libertar um lugar, chama a função signal para desbloquear T1, T1 ainda precisará de readquirir o trinco para avançar na secção crítica e ocupar o lugar que ficou livre.

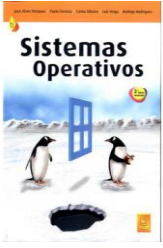
No entanto, caso T3 tente obter o trinco nesse momento, é possível que o trinco seja obtido por T3 (que conseguirá assim ocupar o lugar livre) antes de T1 o obter; caso isso aconteça, T1 verificará que a condição livres==0 ainda se verifica, logo voltará a bloquear-se na variável de condição.

Exercícios (22)

Caderno de exercícios disponível em:

Exercícios de Sistemas Operativos

Com algumas resoluções de exercícios propostos no livro Sistemas Operativos, 2º Edição, FCA 2012.



Paulo Ferreira, Carlos Ribeiro, José Alves Marques, Rodrigo Rodrigues, Luís Veiga,
Sistemas Operativos,
FCA, 2ª Edição, 2012

QUESTÕES?

Daniel Castro
dcastro@ualg.pt

